



A U R A

LUXURY SALON

Technical Development Workflow & Backend Integration Blueprint

Version: 1.2

Target: Full Stack & Backend Developers

Date: June 21, 2026

1. Full Project Development Workflow

Transitioning the Aura Luxury Salon website from a frontend React prototype to an enterprise-grade, production-ready system requires a structured developer workflow. This workflow ensures database integrity, secure user authentication, and high-performance data delivery for the AI Concierge.

1.1 Git Branching Strategy & Lifecycle

To ensure coordinate code integration, developers must follow a standard feature branch workflow:

- main: The production branch. Locked and protected. Only modified via Pull Requests from staging.
- staging: The pre-production testing branch. Used to compile, build, and run integration testing.
- feature/*: Short-lived developer branches (e.g., feature/booking-api, feature/user-auth). Merged into staging after passing unit tests.

1.2 Full-Stack State Flow Lifecycle

The diagram below details the transaction and state sequence between the frontend, the Express backend, and the Database during a booking action:

1. Client triggers checkout with a list of selected service IDs, a date, slot, and stylist.
2. Frontend transmits the payload along with the JWT bearer authorization token to the backend API.
3. Backend authenticates the token, checks for service conflicts in the database, and creates a pending record.
4. Upon successful validation, it commits the booking transaction to the database, decrements the salon queue times, and returns the ticket receipt.

2. Database Schema Specifications

A relational SQL database (e.g., PostgreSQL) is recommended to manage relational keys between Users, Salons, Services, Bookings, and Reviews.

2.1 Table: users

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(150)	UNIQUE, NOT NULL	User email (login)
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
role	VARCHAR(20)	DEFAULT 'customer'	'customer', 'partner', 'admin'
glowpass_tier	VARCHAR(20)	DEFAULT 'Glow'	'Glow', 'Glow+', 'Glow Elite'
diagnostics	JSONB	NULL	Face scan skin results

2.2 Table: salons

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique salon identifier
name	VARCHAR(150)	NOT NULL	Business salon title name
city	VARCHAR(50)	NOT NULL	City name (e.g. Bangalore)
rating	DECIMAL(2,1)	DEFAULT 0.0	Average client rating score
queue_length	INTEGER	DEFAULT 0	Number of active clients in queue

2.3 Table: bookings

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique booking ticket code
user_id	UUID	REFERENCES users(id)	Reserved customer reference
salon_id	UUID	REFERENCES salons(id)	Reserved clinic salon reference
final_price	INTEGER	NOT NULL	Total amount paid (incl. tax, disc.)
status	VARCHAR(20)	DEFAULT 'Confirmed'	'Confirmed', 'Completed', 'Cancelled'

3. REST API Interface Specifications

The API layer handles authentication tokens, data validation, catalog queries, and transactions. It acts as the gateway to retrieve data securely.

3.1 POST /api/auth/login

Authenticates credentials and returns a secure JWT login token.

```
// Request Body
{
  "email": "customer@aura.in",
  "password": "securePassword123"
}

// Response Payload (200 OK)
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "usr-194",
    "name": "Ridhi Maliah",
    "role": "customer",
    "glowpass_tier": "Glow+"
  }
}
```

3.2 GET /api/salons

Retrieves a filtered list of clinics with start pricing and GPS tags.

```
// Query String parameters
GET /api/salons?city=Bangalore&category=hair&search=balayage

// Response Payload (200 OK)
[
  {
    "id": "biz-102",
    "name": "Lotus Styling Lab",
    "city": "Bangalore",
    "rating": 4.8,
    "priceCategory": "$$$",
    "startingPrice": 1200,
    "coordinates": { "lat": 12.9721, "lng": 77.5951 }
  }
]
```

4. Frontend State to API Transition Guide

To transition the React frontend from simulated data (memory) to the API endpoints, the backend developer must replace local state setters with async fetch requests.

4.1 App.jsx: Replace DB Seeding with API Fetches

```
// BEFORE (Local Memory Seeding):
import { generateDatabase } from './data/dataGenerator';
const [salons, setSalons] = useState(() => generateDatabase(2200));

// AFTER (Async API Integration):
const [salons, setSalons] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  const fetchSalons = async () => {
    setLoading(true);

    const res = await fetch(`/api/salons?city=${currentCity}&search=${searchTerm}&category=${activeCategory}`);
    const data = await res.json();
    setSalons(data);
    setLoading(false);
  };
  fetchSalons();
}, [currentCity, searchTerm, activeCategory]);
```

4.2 CheckoutModal.jsx: Connect to API Checkout Booking

```
// AFTER (Persisting Booking transaction):
const handleConfirmBookingSubmit = async () => {
  const payload = {
    salon_id: bookingDetails.salon.id,
    services: bookingDetails.items.map(s => s.id),
    stylist_name: bookingDetails.stylistName,
    booking_date: bookingDetails.date,
    booking_time: bookingDetails.time,
    coupon_code: appliedCoupon
  };
  const token = localStorage.getItem('AURA_JWT_TOKEN');
  const res = await fetch('/api/bookings', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`
    },
    body: JSON.stringify(payload)
  });
  if (res.ok) {
    const ticket = await res.json();
    onConfirmBooking(ticket);
  }
};
```